CISC 322 Assignment 1

WezTerm Conceptual Architecture

October 9, 2026

Team Terminal 27

Francois Olivier: 23kmd5@queensu.ca
Simon Deveaux: 23pjb4@queensu.ca
Joshua Lang: 22WJK1@queensu.ca
Ali Senel: 22VG51@queensu.ca
Affan Farooq: 22maf@queensu.ca
Kay Yan: k.yan@queensu.ca
Liam Wu: 24pjp@queensu.ca

Abstract

This report presents the conceptual architecture of WezTerm, an open-source, GPU-accelerated terminal emulator and multiplexer written in Rust, as recovered from its official documentation. We find that WezTerm is best described as a layered architecture with four layers: presentation, session, terminal and system. This structure is combined with a client–server relationship between the user interface and a multiplexer, and implicit invocation through user-defined Lua event handlers. The central design decision is the multiplexer and its domain abstraction, which lets one interface present local, remote, WSL and serial sessions and lets sessions outlive the windows that display them. We describe WezTerm's major components and their interactions, how the architecture supports evolution, its control and data flow, and the concurrency between panes, the multiplexer and the interface. We then discuss the implications for dividing work among developers and illustrate two use cases with sequence diagrams. We conclude that the architecture's main strengths are extensibility and responsiveness, at the cost of a few central components on which nearly every other part depends. The report ends with lessons learned and an account of our collaboration with an AI teammate.

Table of Contents

Abstract **2**
1 Introduction **3**
2 Functionality and Conceptual Architecture **3**
3 System Evolution and Architectural Evolvability **6**
4 Control and Data Flow **7**
5 Concurrency **7**
6 Implications for Division of Responsibilities Among Developers **9**
7 Use Cases and Sequence Diagrams **10**
8 Conclusion **13**
9 AI Collaboration Report **14**
References **16**

1 Introduction

WezTerm is an open-source, cross-platform terminal emulator and multiplexer written in Rust [1]. Like any terminal emulator, it gives command-line programs a window to run in [3], but it also renders text on the GPU, manages windows, tabs and split panes through a built-in multiplexer, connects to remote machines as if they were local tabs, and lets users reconfigure almost all of its behaviour through a Lua script [5]. This combination makes it a useful case study in software architecture as a simple tool turns out to be a set of cooperating subsystems with many responsibilities and interfaces.

This report presents the conceptual architecture of WezTerm, that is, its abstract structure as recovered from the project's documentation rather than from its source code. Our sources were the official WezTerm documentation, in particular the pages on features, multiplexing, configuration, the Lua API and the command-line interface. We identified candidate components from the concepts the documentation describes (for example windows, tabs, panes, domains and the mux server), grouped them by responsibility, and traced how a key press and a program's output move between them. The concrete architecture, recovered from the source code, will be the subject of our next report.

We conclude that WezTerm is best described as a layered architecture, with presentation, session, terminal and system layers, combined with a client–server relationship between the user interface and the multiplexer, and implicit invocation through Lua event handlers. The multiplexer and its domain abstraction are the central design decision: they let the same interface display local, remote, WSL and serial sessions, and let sessions outlive the windows that show them.

The report is organized as follows. Section 2 describes WezTerm's functionality, architectural styles, major components and how they interact. Section 3 discusses how the architecture supports evolution, Section 4 the control and data flow, and Section 5 the concurrency present in the system. Section 6 considers the implications for dividing responsibilities among developers, and Section 7 presents use cases through sequence diagrams. Section 8 concludes, and Section 9 reports on how our group worked with an AI teammate.

2 Functionality and Conceptual Architecture

2.1 What WezTerm Does

WezTerm is a cross-platform terminal emulator and multiplexer written in Rust by Wez Furlong [1]. It runs on Linux, macOS, Windows, FreeBSD and NetBSD [2]. Like any terminal emulator, it gives command-line programs a window to run in.

What sets WezTerm apart is what it builds on that base. It renders text on the GPU, with full font shaping that supports ligatures, colour emoji and fallback fonts [4]. It includes a built-in multiplexer, so a single instance manages many windows, tabs and split panes, and sessions can outlive the window that displays them, much like tmux or GNU screen [5]. It can connect to other machines over SSH, Unix sockets and TLS, into the Windows Subsystem for Linux, and to serial devices, and it presents each of these as ordinary tabs and panes [5], [6]. Finally, almost all of its behaviour is set in a Lua script, .wezterm.lua, which defines appearance and key bindings and can register handlers for events. This configuration reloads automatically when the file changes [7]. Features such as copy mode, quick select, search, clickable hyperlinks and inline images are built on top of these core capabilities [4].

2.2 Architectural Style

Based on the documentation, we describe WezTerm's conceptual architecture as mostly layered, with a client–server relationship at its centre and invocation used for configuration and scripting. Figure 1 shows the resulting structure.

The layers, from top to bottom, are: a ***presentation layer***, which deals with windows, input and drawing; a ***session layer*** (the multiplexer and its domains), which owns the logical model of windows, tabs, panes and decides where programs run. Then there is a ***terminal layer***, which turns byte streams into screen state, and a ***system layer*** (pseudo-terminals and processes), which talks to the operating system. The principal data path follows this layering, while the Configuration and Lua Engine provides cross-cutting services to multiple layers. This is what allows the same GUI to display a local shell, a remote server or a serial port without any change.

The client–server relationship comes from how the multiplexer is used. The documentation explains that WezTerm creates a default local domain when it starts, but it can also start or connect to additional multiplexer domains and attach their windows and tabs to the native GUI [5].

Implicit invocation appears in the Lua engine. Components announce events, for example that a window's status bar should be refreshed. The component that raises the event does not need to know who is listening [7].

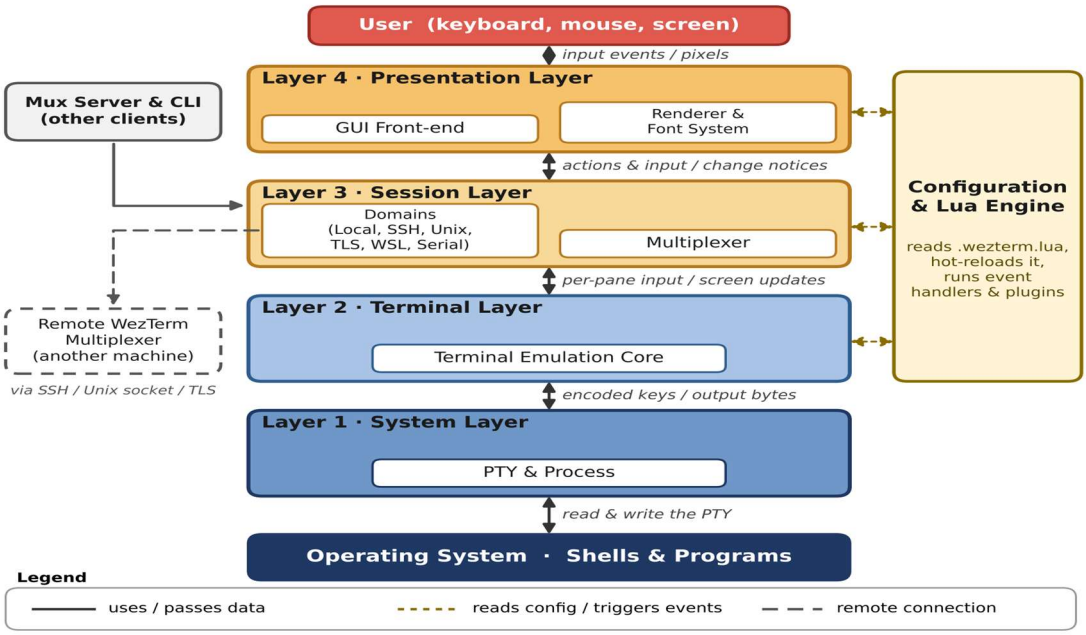


Figure 1: High-level conceptual architecture of WezTerm

2.3 Major Components

GUI Front-end. The GUI owns the native window on each platform and is the user's point of contact with the system. It receives keyboard and mouse input and looks it up against the configured key and mouse bindings [9]. A bound input triggers a WezTerm action, such as opening a tab, splitting a pane or entering copy mode; anything else is forwarded to the active pane. The GUI also draws the tab bar and

overlays such as the launcher and command palette. It holds no terminal state of its own and simply displays what the multiplexer provides.

Renderer and Font System. This component turns screen state into pixels. It locates fonts, shapes text into glyphs, rasterizes and caches those glyphs, and draws the result through a graphics back end [11]. The front_end setting selects OpenGL, WebGPU or a software renderer [10]. Keeping rendering behind its own interface lets WezTerm use different graphics technologies on different platforms without affecting the rest of the system.

Multiplexer. The multiplexer is the hub of the architecture. It maintains the hierarchy of workspaces, windows, tabs and panes, tracks which of them are active, and notifies other components when panes change or produce output. The Lua API mirrors this model directly through its MuxWindow, and Pane objects [7]. Because the multiplexer, rather than the GUI, owns sessions, those sessions can stay after a window closes and be attached again later.

Domains. A domain is a separate set of windows and tabs, and it represents where programs run [5]. The local domain spawns programs on the user's own machine. SSH, Unix and TLS domains connect to a WezTerm multiplexer running on another host or behind a socket, while WSL and serial domains reach the Windows Subsystem for Linux and serial devices [5], [6]. Every domain presents the same interface to the multiplexer, so supporting a new kind of connection means adding a domain rather than changing the GUI.

Terminal Emulation Core. For each pane, this component parses the incoming byte stream, interprets control and escape sequences [3], and maintains the screen grid, cursor position, text attributes and scrollback. It also performs the reverse job of encoding key presses into the sequences programs expect, including newer schemes such as the kitty keyboard protocol [12]. It is independent of both how the screen is drawn and where the bytes come from.

PTY and Process Layer. Local programs run attached to a pseudo-terminal (PTY), which the operating system presents to them as a terminal device [3]. This layer creates the PTY, launches the configured shell or program, moves bytes in both directions, and reports when the process exits.

Configuration and Lua Engine. This component loads .wezterm.lua, exposes the wezterm Lua module, watches the file for changes and reloads it, and runs user handlers when events fire [7]. It also loads plugins written in Lua [13]. Nearly every other component consults it: the GUI for key bindings and appearance, the renderer for fonts and colours, and the multiplexer for domains and default programs. For this reason it sits beside the layers in Figure 1 rather than inside one of them.

Mux Server and CLI. The wezterm-mux-server program runs the multiplexer without a GUI, so sessions can live on a remote host or in the background [5]. The wezterm cli subcommands let users and scripts list, spawn, split and send text to panes in a running instance [8]. Both use the same multiplexer interface as the GUI as they are additional clients and hosts, not separate systems.

2.4 How the Components Interact

Three everyday interactions show how the parts work together. In the input path, the user presses a key and the GUI receives it. The GUI checks the binding through the configuration engine. If the key is bound to an action, the action is carried out. Otherwise the key goes to the multiplexer, which routes it to the active pane; the terminal core encodes it, and the pane's domain delivers it either to the local PTY or

across the connection to a remote program. In the output path, a program writes output and the PTY layer passes the bytes to that pane's terminal core, which updates the screen model. The multiplexer then notifies the GUI that the pane has changed, and the GUI asks the renderer to redraw. For a remote domain, the updates arrive from the remote multiplexer instead and when the connection is slow, WezTerm can predict the result of a key press to hide the delay [5].

Finally in the configuration path, editing .wezterm.lua causes the Lua engine to reload it and the other components to apply the new settings without restarting. The GUI also raises events such as periodic status updates, and the user's handlers can call back into the multiplexer through the Lua API, for example to spawn a new window [7].

Together, these interactions highlight WezTerm's central design decision where the GUI never needs to know whether a pane's program runs locally inside WSL, on a remote server or over a serial line, because the multiplexer and the domain abstraction hide that detail. The same decision is what lets sessions survive a closed window and lets the command-line tool control panes. The trade-off is that the multiplexer and the Lua engine are depended on by almost every other component, so changes to their interfaces have wide-reaching effects.

3 System Evolution and Architectural Evolvability

3.1 Separation of Logical and Presentation Responsibilities

WezTerm is built using isolated modules known as crates. By keeping the pure logical state of the terminal completely separate from the graphical interface, the system can easily evolve over time. For example, developers can swap out rendering backends or add support for new operating systems without ever touching the core terminal logic. This separation also means the core state machine is highly testable in isolation, which fulfills a key architectural requirement.

3.2 Multiplexer Architecture

A major way the system evolves is through its client and server multiplexer model. Instead of tightly linking the user interface and the terminal state, WezTerm separates them entirely. This specific architectural choice naturally allows the application to expand from a simple local terminal into a distributed remote session manager. If the creators had chosen a simpler monolithic approach, decoupling those sessions later would have been incredibly difficult.

3.3 User Driven Lua Scripting

Instead of hardcoding hotkeys or window management rules, WezTerm offloads a massive amount of its evolutionary potential to the user space through an embedded Lua runtime. The system exposes its internal events to Lua so users can dynamically change how the terminal behaves on the fly. This allows the system to evolve organically based on user needs without requiring official software updates or recompilation.

3.4 Protocol Abstraction Layers

Finally, the system evolves to support modern terminal features through strict abstraction layers. When adding a new feature like an image rendering protocol, the required changes are localized strictly to specific parsers and the user interface renderer. This clear division of responsibilities makes sure that

developers working on operating system integration do not step on the toes of developers working on core parsing, which keeps the system highly modular.

4 Control and Data Flow

4.1 Overview

When a user types in WezTerm, the GUI frontend catches the keystroke and checks it against the user's key bindings. If it isn't a shortcut, it goes to the **Mux**, which tracks all windows, tabs and panes and selects the active pane to receive it. The pane sends the keystroke through the pseudo-terminal (PTY), the pipe that carries it to the shell. The shell runs the command and sends that text back through the same pipe as **raw bytes**. The terminal emulator core reads those bytes, interprets special codes (such as colours, cursor moves) and keeps track of what the screen should look like. Finally, the renderer draws the updated screen using the user's fonts and settings.

4.2 Control Flow and Data flow

The GUI handles user input. Background I/O activity processes shell output independently of the GUI and requests a redraw when the screen state changes. This keeps the window responsive while a command is still running. Key bindings and settings come from the user's configuration (Lua scripts), which is the GUI frontend reads when handling input. Splitting input handling, shell output, and drawing into separate parts means a slow command can't freeze the window, and each part can be changed without rewriting the others. The data follows through four forms: keystrokes → bytes → screen contents → pixels.

5 Concurrency

There are a lot of things that happen inside a terminal emulator at once. For example, they need to handle programs producing output, the user typing a prompt or the screen being redrawn, all at the same time. WezTerm's architecture separates these processes so that no single slow task will make the entire terminal freeze. An obvious source of concurrency comes from the panes.

5.1 Concurrent Panes

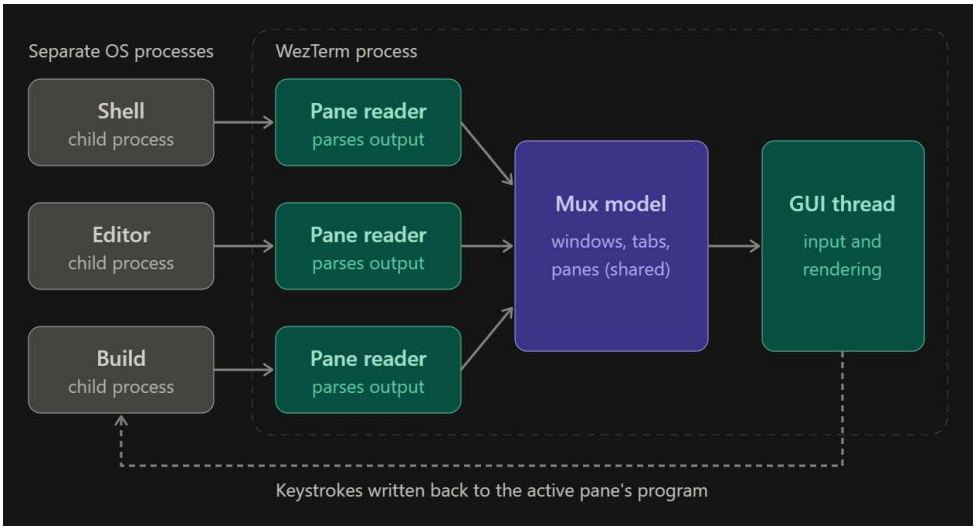


Figure 2: Each pane's output is read concurrently with the GUI so a program producing heavy output cannot block user input [15]

Each pane has its own child program; this could range from a shell, a text editor like vim or some build process as shown in Figure 2. These programs are separate operating system processes, so they're concurrent with WezTerm and concurrent with each other, regardless of which pane the user is accessing. The output from each pane's program is read and interpreted in the background, even while the user works in a separate one. The terminal emulator reads each program's output and interprets its escape sequences to update the display [3]. Conceptually each pane can be thought of as having its own reader that collects output, interprets it and updates the contents of its pane. This all passes while the interface stays free to respond to the user. We see this as a pipe-and-filter style flow where each stage transforms the data before passing it along to the next. The benefit of this flow is program responsiveness; if a program were to flood the terminal with thousands of lines, it wouldn't be able to block the user from typing or freeze the window.

5.2 Shared Multiplexer and Remote Sessions

This ties into the next point of concurrency, the multiplexer. According to WezTerm's official documentation, the multiplexer "manages the set of running programs into panes, tabs, windows and workspaces" [14]. As both the background readers and the GUI depend on this model, it works similarly to the repository style discussed in class. The core idea being one central store that several activities read from and write to. That also means access to it must be coordinated, or two activities could leave it in an inconsistent state. Beyond a single machine, WezTerm organizes its sessions into domains, each domain is a distinct set of windows and tabs [5]. With an SSH domain, WezTerm starts a multiplexer daemon (background process) on the remote host and connects to it [5]. The documentation also notes that the multiplexer may not be connected to a GUI at all [14], so the multiplexer and the window you see are separate pieces that can run independently of each other. This is essentially a client-server arrangement, which can allow remote sessions to keep running in the background. It also means that the design has to cope with network delays without making the local window feel slow. Finally, WezTerm's Lua configuration adds one more consideration. The doc warns that the config file "can be evaluated multiple times in various contexts" [14], so configuration code can't assume it only runs once.

5.3 Implications for Quality Attributes

All together, these choices line up with the qualities users care about most in a terminal. Separating output from the interface supports performance and responsiveness. Running the multiplexer independently of the GUI supports availability and can let sessions continue running. For developers, the shared multiplexer model also acts as a natural dividing line. Making it so that people working on rendering, and people working on terminal parsing can mostly stay out of each other's way, but anyone changing the shared model needs to be careful, because every other part depends on it.

6 Implications for Division of Responsibilities Among Developers

WezTerm's architecture supports dividing development responsibilities according to major subsystems and their stable interfaces.

6.1 Subsystem Ownership

One group of developers can focus on the terminal model, including escape-sequence interpretation, screen state, scrollback, selection, and terminal behaviour. Because this part is independent of the windowing system, it can be developed and tested without depending on a specific operating system or graphical interface.

Another group can own the graphical front end. Their responsibilities include creating native windows, processing keyboard and mouse input, rendering terminal contents, handling fonts and the clipboard, and presenting tabs or panes to the user. The graphical layer should communicate with the terminal model through clearly defined interfaces rather than duplicating terminal behaviour.

A third responsibility area is multiplexing and session management. These developers would handle domains, windows, tabs, panes, workspaces, process creation, and connections to local or remote sessions. This includes coordinating local processes as well as SSH, Unix-domain, and other remote multiplexer connections. The multiplexer must expose a stable interface to the GUI so that the same user interface can work with different kinds of sessions.

Configuration, command-line features, platform integration, documentation, and system testing form additional responsibilities. Developers in this area would maintain configuration loading, Lua events, command-line commands, operating-system-specific adaptations, build support, and user documentation. These responsibilities are cross-cutting because changes to configuration or platform behaviour can affect the terminal, multiplexer, and GUI layers.

6.2 Coordination Between Teams

This division allows developers to work in parallel and limits the number of components affected by a change. For example, adding a new terminal escape sequence should primarily affect the terminal-model developers, while adding a new remote session type should primarily affect the domain and transport developers. However, the boundaries also create coordination requirements. Interfaces for terminal events, pane operations, configuration values, and client-server communication must be documented and reviewed jointly. Features that cross several boundaries, such as reconnecting a remote session or adding a new keyboard action, require collaboration between the relevant subsystem owners.

Concurrency makes coordination especially important. GUI processing must remain responsive while terminal input/output, process management, and remote communication occur in the background. Developers therefore share responsibility for message ordering, synchronization, cancellation, error handling, and reconnection behaviour. Each subsystem should have focused unit tests, while integration tests should verify that the GUI, terminal model, multiplexer, and process or network layers work together correctly.

6.3 Ownership and Review Model

Overall, the architecture encourages a primary owner and a secondary reviewer for each subsystem rather than completely isolated developers. This preserves accountability while reducing knowledge silos and single points of failure. Regular interface reviews, shared integration testing, and documentation are necessary to ensure that independent subsystem work continues to form one coherent terminal application.

7 Use Cases and Sequence Diagrams

This section follows two use cases through the components of Figure 1 using UML sequence diagrams. We chose them because together they exercise most of the architecture. The first is the most common thing a user does in a terminal, and the second shows the multiplexer and domain abstraction that we identified as the central design decision. The numbers in parentheses below refer to the message numbers in each figure.

7.1 Use Case 1: Typing a Key and Seeing the Output

In this use case the user types a character in a local pane, for example a command at a shell prompt and the shell's output appears on screen. The flow has two halves, an input path and an output path (Figure 3).

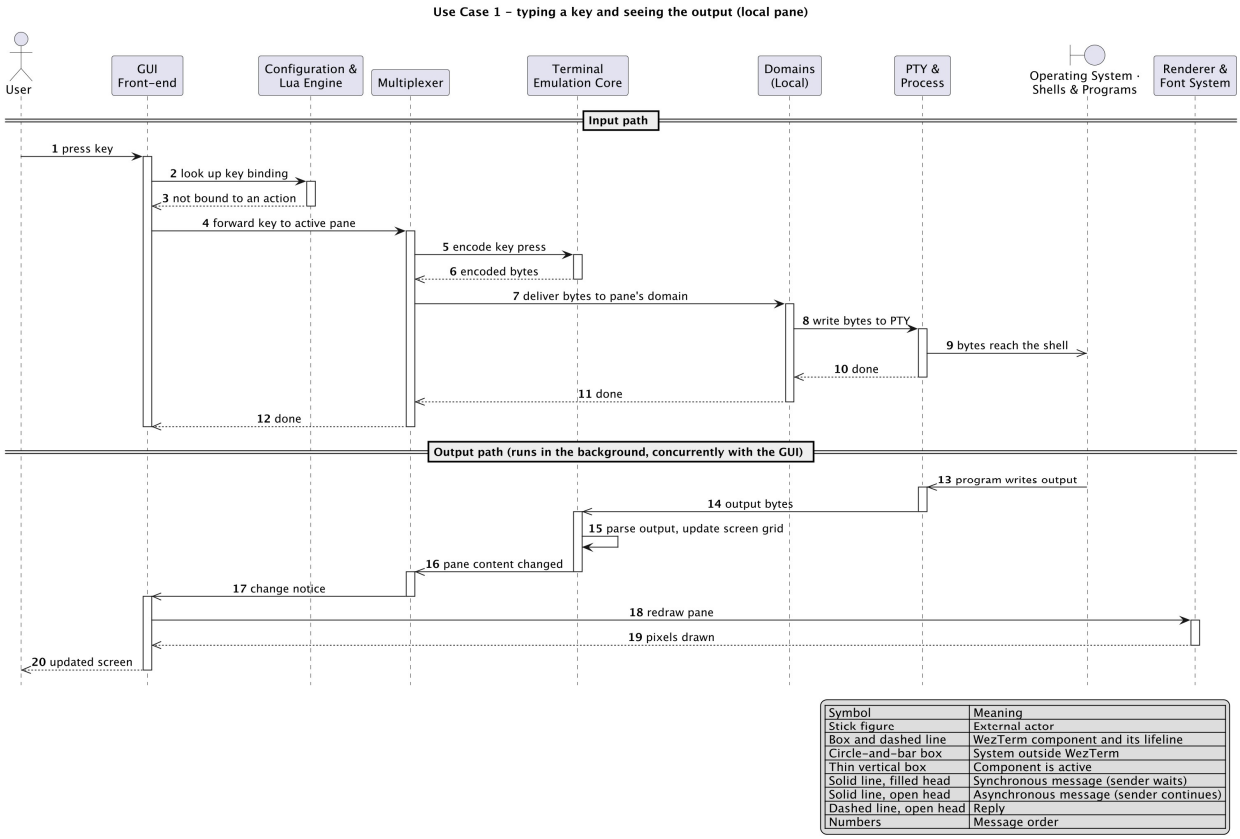


Figure 3: Sequence diagram for Use Case 1, typing a key and seeing the output in a local pane.

In the input path, the GUI Front-end receives the key press (1) and asks the Configuration and Lua Engine whether the key is bound to an action [9] (2). It is not, so the GUI forwards the key to the Multiplexer (4), which routes it to the active pane. The Terminal Emulation Core encodes the key into the byte sequence that programs expect [12] (5-6). The Multiplexer then hands the bytes to the pane's domain (7), here the local domain, which writes them to the PTY and Process layer (8). The PTY passes them to the shell (9). If the key had been bound, for example to a split pane shortcut, the GUI would have carried out the action itself at step 3 and the key would never have left the GUI.

The output path starts when the program writes its response (13). The output-handling path runs independently of the GUI., which is why its messages are asynchronous. The PTY layer passes the bytes to the pane's Terminal Emulation Core (14), which interprets the escape sequences and updates the screen grid (15) [3]. The Multiplexer then finds out that the pane has changed (16) and sends a change notice to the GUI (17). The GUI asks the Renderer and Font System to redraw (18), and the user sees the updated screen (20).

This use case shows three things about the architecture. First, the input path moves from the presentation layer down to the system layer of Figure 1, and the output path goes back up, so the work is divided by layer as the architecture intends. Second, the output path is a pipeline in which each stage transforms the data and passes it on (bytes, then screen contents, then pixels). Third, because output is read and processed separately from the GUI, a program that floods the terminal cannot stop the window from accepting input.

7.2 Use Case 2: Connecting to a Remote Domain and Running a Command

In this use case the user connects to a remote machine through an SSH domain and runs a command in it (Figure 4). It shows how the same interface works for a pane whose program is not on the local machine.

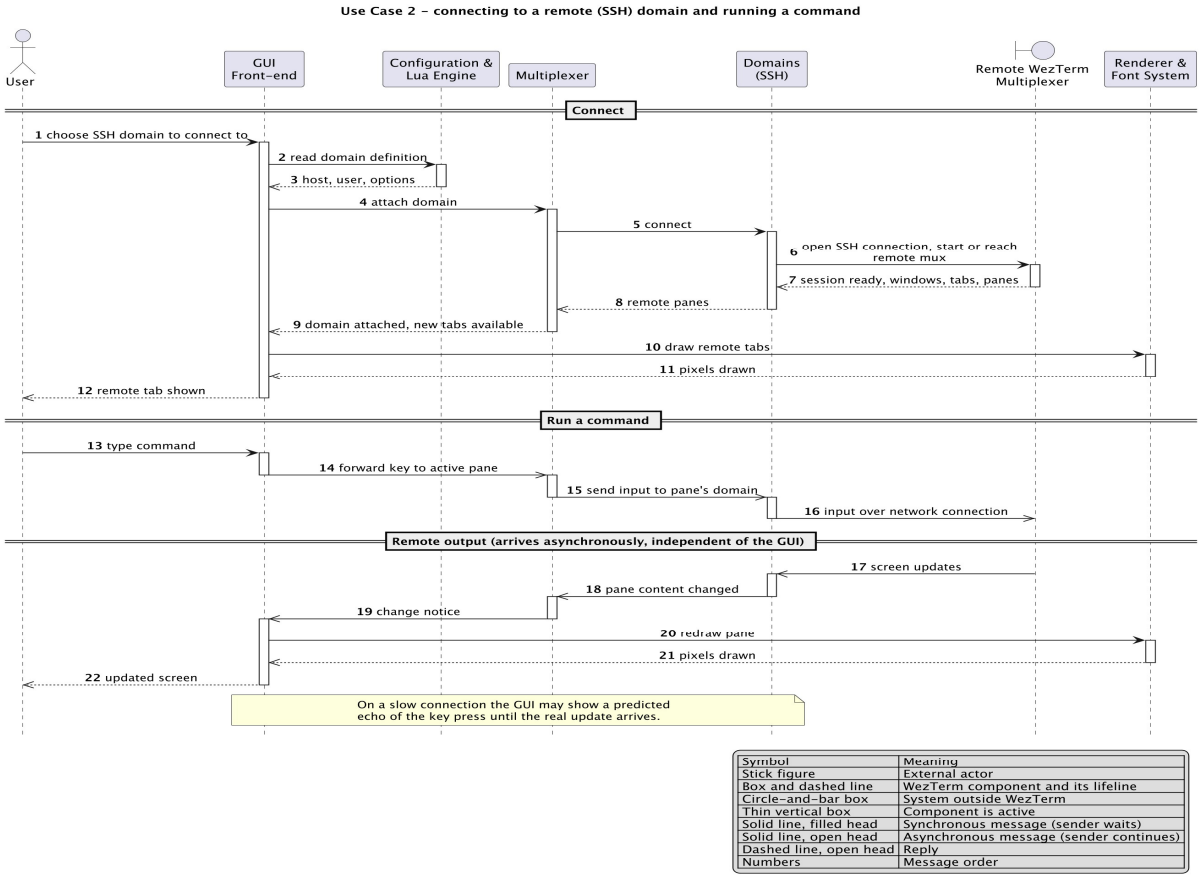


Figure 4: Sequence diagram for Use Case 2, connecting to a remote SSH domain and running a command.

When connecting, the user chooses an SSH domain in the GUI (1). The GUI reads its definition from the Configuration and Lua Engine [7] (2-3) and asks the Multiplexer to attach it (4). The Multiplexer asks the SSH domain to connect (5), and the domain opens an SSH connection and starts or reaches a multiplexer on the remote host (6) [5]. The remote multiplexer replies with its windows, tabs and panes (7), which the domain passes up as new remote panes (8). The Multiplexer reports that the domain is attached (9), the GUI draws the remote tabs through the Renderer and Font System (10-11), and the user sees the remote tab (12).

When the user types a command, the key travels from the GUI to the Multiplexer to the SSH domain and out to the remote multiplexer (14-16). These are asynchronous messages, because the GUI cannot wait for a network round trip without freezing. Output comes back the same way, the remote multiplexer sends screen updates to the domain (17), the domain tells the Multiplexer that the pane changed (18), and the Multiplexer notifies the GUI, which redraws (19-22). On a slow connection WezTerm can display a predicted echo of the key press until the real update arrives [5], which Figure 4 shows as a note.

Compared with Use Case 1, the only steps that differ are those involving the domain. The GUI and the Multiplexer send and receive the same messages as for a local pane, which is the benefit of the domain

abstraction. The local WezTerm acts as a client of a multiplexer running on another host, and communication with it is asynchronous. The cost is that the design must hide network delay without making the local window feel slow.

8 Conclusion

8.1 Summary

From the documentation alone, WezTerm emerges as a carefully separated system. A presentation layer handles windows, input and drawing. A session layer, made up of the multiplexer and its domains, owns the model of windows, tabs and panes and decides where programs run. A terminal layer turns byte streams into screen state. Finally a system layer connects to the operating system through pseudo-terminals. A configuration and Lua engine sits beside these layers and is consulted by nearly all of them.

The main strength of this architecture is the abstraction provided by the multiplexer and its domains. Because the user interface never needs to know where a pane's program is running, WezTerm can add new kinds of connection without changing the interface, keep sessions alive after a window closes, and let the command-line tool control the same panes as the GUI. Separating output processing from the interface also keeps the terminal responsive under heavy output, and the Lua engine allows users to extend the system without recompiling it.

These benefits come with costs. The multiplexer and the Lua engine are dependencies of almost every other component, so changes to their interfaces affect the whole system and require coordination between developers. Remote sessions must hide network latency, and shared session state must be protected from concurrent access. Finally, because this report is based on documentation, several of our components are interpretations rather than named parts of the code; our next report will test these against the concrete architecture recovered from the source.

8.2 Lessons Learned

Recovering an architecture from user documentation is an act of interpretation. WezTerm's documentation is written for users configuring the terminal, not for developers studying its structure, so components such as the terminal emulation core had to be inferred from features and from the objects exposed by the Lua API. We learned to state clearly which parts of our architecture the documentation supports directly and which are our own grouping.

We also learned that a report written by several people needs shared vocabulary from the start. Our sections were drafted in parallel, and the same component appeared under different names, such as "Mux" or "multiplexer". Agreeing on a single component list and diagram before writing would have saved integration effort. From our group experience we decided to split each task up and do them separately. Then all of them were sent to both the presenters and the group leader so that presentations can be started and the format can be fixed together. We discussed all the information through group chat and calls, and all came to a final conclusion that we should be together to complete these assignments as certain tasks are built on top of each other and combining them all in a single word document is fairly difficult.

When all the work was completed we decided to pair up and proofread each others work. This gave all the work an overall proofread through someone else. We found this strategy strong and an important lesson was learned to how we should be proofreading for the future assignments.

9 AI Collaboration Report

Our group worked with an AI teammate throughout this deliverable. This section describes which model we used, which tasks we gave it, how we interacted with it, how we checked its work, and how much of the final report it produced.

9.1 Model Selection

Our main AI teammate was Claude, accessed through Claude.ai. We chose it because it can work with long documents in a single conversation, which let us give it the assignment specification, last year's example report and our group website together. This then gave a basic guide for formatting, what the assignment is searching for, how we can succeed, and an overall strategy on how the work can be split up evenly.

9.2 Task Assignment

We assigned the AI tasks where speed and consistency mattered and kept architectural judgement with the group. Per task the AI was asked to, (1) create an overall diagram given the information prompted of the structure. Split the overall texts into easy to follow paragraphs and to finally check to make sure all the references are properly made. (2) Ai was given a prompt including all the basic system parts of WezTerm and the architecture and was asked to take out overlapping information. This gave an overall easy to follow document of important information which was then read by the user. (3) Claude was used to highlight important information in documents and was given a prompt to explain it in simple terms. This overall increased the speed of analyzing information given a document, and gave a simple insight into complex topics. (4) Claude was used to create a diagram, given the information that the user read about and asked Claude to follow. Simply used Claude as it is much quicker to draft images instead of photo editors or Canva. Claude was also given a prompt to read over the paragraphs and search for confusing sentences, improper formatting and proper wording. (5) Our AI teammate was only used to divide up the information once it was all written out. This gave proper formatting and allowed it to flow seamlessly into the final word document. (6) PlantUML was used to create the images, then AI was used to aid in synthesize the diagrams. AI was also used to format input as well as help render the diagram properly.

9.3 Interaction Protocol and Prompting Strategy

Integration and formatting were handled in a single conversation, so that the AI kept the full context of the report. Each teammate used the AI for their own section and kept all information given by the AI in a single shared document. Our prompting followed a consistent pattern. We provided context first (Typically direct sources, or a brief idea of our summary) before asking for output. We also asked for an outline before asking for full text, so we could correct the direction early. Finally, since we had to combine all the tasks at the end, we prompted Claude to help with formatting. This included indentations, proper reference documenting, and an overall table of contents. This strongly helped as it aligned through our overall strategy and made combining the tasks super time efficient. Changes were then made through short, specific follow-up requests, such as adding numbered subsections or redrawing a figure to match an example image, rather than regenerating whole sections.

9.4 Quality Control

We treated all AI output as a draft. Citations in the AI-drafted text point to specific WezTerm documentation pages so that they can be checked. Quotations were checked word for word against their sources and during integration this confirmed that the quotations in Section 5 come from the documentation of the wezterm.mux module, which led us to correct their reference numbers. We were all responsible for each section to read and edit the AI's draft before it was merged, and rendered pages were inspected visually after each formatting change. The review step was mostly done by us members as AI has a strong tendency to simply agree to users as a form of being correct. Therefore, we believed that it would be stronger to each group up with the adjacent task and proofread in pairs.

9.5 Reflection on Human–AI Team Dynamics

The AI was most valuable for structure and consistency, turning a requirement into an outline. This allowed terminology and formatting uniform across seven authors, and producing clean diagrams quickly. It was least reliable where interpretation was required, because it produces fluent, plausible text whether the sources fully support it, so its architectural claims needed the same proofread we would give a teammate's. Using the AI separately in different sections also contributed to the inconsistent component names. In future deliverables we will share one component glossary and one diagram with the AI from the start, and keep the human reviewer for each section responsible for its final content.

9.6 Quantitative Impact

Part of the report	AI contribution
Section 1-6 (about 3000 words, 30-40% of Sections 1–6)	Parts drafted by AI, proofread by members. AI used for basic summary and formatting
Figure 1	Produced by AI
Figure 2	Produced by AI
Figures 3+4	Used PlantUML to create and used AI to help synthesize and render the diagram
Abstract, Introduction, Conclusion	Drafted about half AI, half members
Table of Contents	Drafted by AI, checked by a group member
Formatting, numbering, references	Done by AI, checked strongly by members and frequently edited by group
Choice of system, architecture decisions, final review	Group only

Overall, about 30-40% of the words in the final report were first drafted by the AI, and all of it was reviewed and edited by group members before submission.

References

[1] WezTerm documentation, home page. https://wezterm.org/index.html Accessed: Oct. 2026.

[2] WezTerm documentation, Download. https://wezterm.org/installation.html Accessed: Oct. 2026.

[3] WezTerm documentation, What is a Terminal?. https://wezterm.org/what-is-a-terminal.html Accessed: Oct. 2026.

[4] WezTerm documentation, Features. https://wezterm.org/features.html Accessed: Oct. 2026.

[5] WezTerm documentation, Multiplexing. https://wezterm.org/multiplexing.html Accessed: Oct. 2026.

[6] WezTerm documentation, Serial Ports & Arduino. https://wezterm.org/serial.html Accessed: Oct. 2026.

[7] WezTerm documentation, Full Config & Lua Reference. https://wezterm.org/config/lua/general.html Accessed: Oct. 2026.

[8] WezTerm documentation, CLI Reference. https://wezterm.org/cli/general.html Accessed: Oct. 2026.

[9] WezTerm documentation, Key Binding. https://wezterm.org/config/keys.html Accessed: Oct. 2026.

[10] WezTerm documentation, front_end. https://wezterm.org/config/lua/config/front_end.html Accessed: Oct. 2026.

[11] WezTerm documentation, Font Shaping. https://wezterm.org/config/font-shaping.html Accessed: Oct. 2026.

[12] WezTerm documentation, Keyboard Encoding. https://wezterm.org/config/key-encoding.html Accessed: Oct. 2026.

[13] WezTerm documentation, Plugins. https://wezterm.org/config/plugins.html Accessed: Oct. 2026.

[14] WezTerm documentation, module: wezterm.mux. https://wezterm.org/config/lua/wezterm.mux/index.html Accessed: Oct. 2026.